

AUDITORIA FORENSE

VitaZen

Free vs Elite

Verificacion funcional completa de planes de suscripcion

Repositorio: github.com/josinesprados-hub/VitaZen | Rama: main

Metodologia: Solo lectura. Evidencias reales del codigo. Cero suposiciones.

Fecha: 11 de julio de 2026

INDICE

1. Resumen Ejecutivo	4
1.1 Veredicto Global	4
2. Arquitectura del Sistema de Suscripcion	5
2.1 Fuentes de diferenciacion entre planes	5
2.2 Constantes y limites definidos	5
2.3 Proteccion contra condiciones de carrera	6
3. Auditoria del Plan Free - 17 Funcionalidades	7
3.1 Dashboard	7
3.2 Check-in	7
3.3 Habitos	7
3.4 Mentor IA (Free)	7
3.5 Memoria de Vida	8
3.6 Tu Evolucion (Insights)	8
3.7 Observaciones	8
3.8 Diario	8
3.9 Notas	9
3.10 Logros	9
3.11 Imperios (5 imperios)	9
3.12 Recomendaciones	9
3.13 Patrones	10
3.14 Cierre Mensual	10
3.15 Notificaciones	10
3.16 Perfil	10
3.17 Ajustes	11
4. Auditoria del Plan Elite - Verificacion de Promesas	12
4.1 Promesas verificadas	12
4.2 Diferenciacion real del contexto del Mentor Elite	12
5. Conexiones Reales Entre Imperios	14

5.1 Conexiones directas (motor de patrones)	14
5.2 Conexiones agregadas (modulos analiticos)	14
5.3 Conexiones NO implementadas	15
6. Placebos Detectados	16
6.1 Detalle de cada placebo	16
7. Funciones Ocultas (No Anunciadas en Elite)	17
8. Vulnerabilidades de Seguridad Identificadas	18
8.1 API de Patrones sin proteccion en servidor	18
9. Componentes con Nomenclatura Enganosa	19
10. Matriz Final Completa	20
11. Veredicto Final y Recomendaciones	21
11.1 Veredicto	21
11.2 Problemas criticos	21
11.3 Problemas menores	21
11.4 Fortalezas observadas	21

1. Resumen Ejecutivo

Esta auditoria forense examina el repositorio completo de VitaZen en la rama main, verificando que cada funcionalidad anunciada para los planes Free y Elite esta realmente implementada en el codigo. El analisis se basa exclusivamente en evidencias reales: archivos fuente, funciones, componentes y rutas API. No se han realizado suposiciones ni interpretaciones sin respaldo en el codigo.

VitaZen es una aplicacion SaaS de seguimiento de salud y bienestar publicada en Google Play y App Store. Su modelo de suscripcion se basa en dos planes: Free (0 EUR/mes) y Elite (5 EUR/mes, internamente denominado PREMIUM). La diferenciacion se controla mediante un campo unico **User.plan** con valores "FREE" o "PREMIUM", gestionado exclusivamente a traves de webhooks de Stripe. No existe middleware que bloquee rutas por plan; toda la diferenciacion se aplica en las rutas API individuales y en los componentes del cliente.

Se han auditado un total de 17 funcionalidades del plan Free, 9 promesas especificas de la pagina Elite, 3 placebos confirmados, 11 funciones ocultas no anunciadas, 5 detectores de patrones cross-imperio, y 7 componentes Premium con nomenclatura engañosa. A continuacion se presenta el veredicto global.

1.1 Veredicto Global

VEREDICTO: D) El sistema necesita una revision antes de publicarse.

Aunque la gran mayoria de las funcionalidades del plan Free estan completamente implementadas (15 de 17) y la mayoria de las promesas Elite tienen respaldo en el codigo (8 de 9), existen problemas que requieren atencion antes del lanzamiento: una promesa de Elite es completamente placebo, tres controles de notificaciones no hacen nada, y un endpoint API de patrones carece de proteccion en el servidor, exponiendo datos de pago a usuarios Free.

Metrica	Total	Implementado	Parcial	No implementado
Funcionalidades Free	17	15 (88%)	2 (12%)	0 (0%)
Promesas Elite	9	8 (89%)	0 (0%)	1 (11%)
Placebos detectados	3	-	-	3
Funciones ocultas	11	11 (100%)	-	-
Conexiones entre imperios	5	5 (100%)	-	-
Vulnerabilidades de gating	1	-	-	1

2. Arquitectura del Sistema de Suscripcion

El sistema de suscripcion de VitaZen se basa en un modelo simple pero robusto. El campo **User.plan** en la base de datos PostgreSQL (Prisma ORM) almacena el estado del plan como un string: "FREE" (por defecto) o "PREMIUM". Este campo se modifica exclusivamente a traves de webhooks de Stripe, nunca por logica de la aplicacion directamente. La propagacion del plan al cliente ocurre a traves del flujo de autenticacion: el servidor devuelve el campo plan en cada sesion, y el contexto AuthContext lo almacena en el estado de React para que todos los componentes puedan consultarlo.

2.1 Fuentes de diferenciacion entre planes

La diferenciacion entre Free y Elite no se aplica en un solo punto, sino que esta distribuida en multiples capas de la aplicacion. En el servidor, varias rutas API verifican el campo **user.plan** y retornan diferentes volumenes de datos, diferentes prompts del sistema para la IA, o aplican truncamientos a la respuesta. En el cliente, los componentes PremiumGate reducen la opacidad del contenido bloqueado al 40% y superponen un enlace a la pagina de suscripcion Elite. Es importante notar que el middleware de Next.js **no** realiza ningun cheque de suscripcion; toda la aplicacion de gates ocurre en rutas y componentes individuales.

Capa	Mecanismo	Archivo de referencia
Base de datos	User.plan: "FREE" "PREMIUM"	prisma/schema.prisma (linea 26)
Servidor - IA	Limites diarios, contexto, prompts	src/lib/limits.ts, src/lib/groq.ts
Servidor - API	Truncamiento de respuestas por plan	src/app/api/life-memory/route.ts
Servidor - IA Chat	Historial, temperatura, tokens	src/app/api/ai/chat/route.ts
Servidor - Threads	Limite de hilos y mensajes visibles	src/app/api/ai/threads/route.ts
Servidor - Insights	Cantidad de insights, comparativas	src/lib/insights.ts
Servidor - Cierre	trimDigestForFree()	src/app/api/monthly-closure/route.ts
Cliente - UI	PremiumGate (40% opacidad)	src/components/ui/PremiumGate.tsx
Cliente - Patrones	Blurred preview del primer patron	src/components/patterns/LifePatternsSection.tsx
Cliente - Timeline	3 grupos de días maximos	src/app/(dashboard)/timeline/page.tsx

2.2 Constantes y limites definidos

Los limites entre planes estan definidos en multiples archivos, no centralizados en un solo lugar. El archivo principal de constantes es **src/lib/stripe.ts** que define PLANS.FREE (15 mensajes IA/dia) y PLANS.PREMIUM (Infinity). Sin embargo, otros limites como la cantidad de hilos, mensajes por hilo, insights, y duracion del historial estan codificados directamente en sus respectivos archivos de ruta API. No existe un archivo unico de constantes de funcionalidad.

Parametro	Free	Elite	Archivo
Mensajes IA/dia	15	Sin limite	src/lib/limits.ts (linea 4)
Historial por conversacion	10 mensajes	30 mensajes	src/app/api/ai/chat/route.ts (linea 111)

Parametro	Free	Elite	Archivo
Hilos visibles	10	100	src/app/api/ai/threads/route.ts (lineas 7-8)
Mensajes por hilo	50	Sin limite	src/app/api/ai/threads/[threadId]/messages/route.ts (linea 7)
Temperatura IA	0.5	0.8	src/app/api/ai/chat/route.ts (linea 148)
Tokens maximos	800	2048	src/app/api/ai/chat/route.ts (linea 149)
Insights semanales	3	5	src/lib/insights.ts (linea 645)
Grupos timeline	3	Todos	src/app/(dashboard)/timeline/page.tsx (linea 235)

2.3 Proteccion contra condiciones de carrera

Un aspecto notable de la arquitectura es el uso extensivo de **pg_advisory_xact_lock** para prevenir condiciones de carrera en operaciones criticas. El checkout de Stripe utiliza un advisory lock por userId para evitar double-checkout. El limite de mensajes IA usa el mismo patron con atomicidad a nivel de transaccion. El completion de habitos emplea SELECT FOR UPDATE a nivel de fila. La reversa de XP al eliminar registros (check-ins, habitos, diario) se ejecuta dentro de transacciones Prisma. Esta arquitectura demuestra un nivel de madurez tecnica inusual para una aplicacion en fase previa a produccion.

3. Auditoria del Plan Free - 17 Funcionalidades

A continuacion se audita cada una de las 17 funcionalidades que la aplicacion ofrece a los usuarios del plan Free. Para cada una se indica el estado de implementacion, los archivos y funciones responsables, y las evidencias especificas encontradas en el codigo. La evaluacion sigue tres categorias: Implementada Completamente, Implementada Parcialmente, o No Implementada.

3.1 Dashboard

ESTADO: Implementada Completamente

El dashboard principal (**src/app/(dashboard)/dashboard/page.tsx**) muestra: un saludo consciente de la hora en zona horaria Madrid, el componente EmotionalHero con el estado emocional en tiempo real, el estado del check-in del dia con la intencion y el emoji de emocion o un boton de check-in, una cuadrricula con los 5 imperios mostrando nivel, racha y barra de progreso XP, un componente PremiumReflection con la cita diaria, la seccion LifePatternsSection con patrones cross-imperio, y un MonthlyClosurePrompt cuando corresponde. Realiza solo 2 llamadas API: GET /api/empire y GET /api/checkin?mode=today. El dashboard esta completamente funcional para usuarios Free.

3.2 Check-in

ESTADO: Implementada Completamente

El sistema de check-in es una de las funcionalidades mas robustas de la aplicacion. La pagina (**checkin/page.tsx**) muestra el resumen del dia, graficos de tendencia de 14 dias (barras mini para emocion, energia, foco y estres), e historial completo con edicion y eliminacion por entrada. El modal CheckInModal (**src/components/checkin/CheckInModal.tsx**) recoge 6 campos: emocion (1-5 slider), energia (1-5), foco (1-5), estres (1-5), intencion (texto obligatorio, max 120 caracteres) y nota (opcional, max 300 caracteres). La ruta API (**src/app/api/checkin/route.ts**) implementa CRUD completo con upsert Prisma usando la restriccion @@unique([userId, date]), proteccion contra carrera con pg_advisory_xact_lock, y otorga +10 XP al imperio mente solo en la primera creacion diaria. La eliminacion revierte el XP en una transaccion. Incluye accesibilidad completa: ARIA roles, focus trap, y soporte de teclado.

3.3 Habitos

ESTADO: Implementada Completamente

El CRUD de habitos (**src/app/api/habits/route.ts**) soporta creacion, edicion, eliminacion y completion. El motor de rachas es sofisticado: es consciente de la frecuencia (diaria, semanal, mensual), protege contra doble-completacion con SELECT FOR UPDATE a nivel de fila, calcula la continuation de racha con un umbral flexible (threshold * 2), y solo incrementa la racha del imperio una vez por dia de Madrid. Cambiar la frecuencia de un habito resetea su racha a 0. La eliminacion de un habito revierte el XP y decrementa la racha del imperio si ese habito fue el que disparo el incremento del dia. No tiene gate de pago: completamente funcional para todos los usuarios.

3.4 Mentor IA (Free)

ESTADO: Implementada Completamente

El mentor IA es completamente funcional para usuarios Free con 15 mensajes diarios. El sistema utiliza el modelo llama-3.3-70b-versatile via Groq. El prompt del sistema para Free (**src/lib/groq.ts**, lineas 7-47) incluye una seccion "EFICIENCIA" que instruye a la IA a ser concisa dado que el usuario tiene mensajes limitados. El contexto que

recibe el mentor Free incluye: 2 check-ins recientes (vs 5 para Elite), 4 rachas de habitos (vs 8), 1 meditacion (vs 5), 1 titulo de diario (vs 3 con contenido), y 1 hilo de conversacion (vs 3). Los parametros de generacion son mas conservadores: temperatura 0.5 (vs 0.8) y max 800 tokens (vs 2048). El componente MentorChat (`src/components/mentor/MentorChat.tsx`) es un chat completo con sidebar de hilos, creacion/archivado/eliminacion/renombrado, contador de limite diario, y auto-generacion de titulo en el primer intercambio.

3.5 Memoria de Vida

ESTADO: Implementada Parcialmente (gating por plan)

La memoria de vida (`src/app/(dashboard)/memoria-de-vida/page.tsx`) y su API (`src/app/api/life-memory/route.ts`) implementan un sistema de timeline con 4 tipos de observaciones: Etapas (periodos mensuales de vida con flavor como calm, growth, intensity), Transiciones (cambios detectados entre etapas), Memorias (momentos reales extraidos de finanzas, diario y check-ins), y Patrones (correlaciones cross-imperio del motor de patrones). Los usuarios Free **solamente ven las Etapas**. Las Transiciones, Memorias y Patrones estan bloqueadas detras de PremiumGate en el UI y truncadas en el servidor (lineas 111-118 de la ruta API: `transitions: []`, `memories: []`, `observations` filtradas a tipo "stage" solamente). La funcionalidad base (timeline de etapas) esta completamente implementada y es funcional.

3.6 Tu Evolucion (Insights)

ESTADO: Implementada Parcialmente (gating por plan)

El motor de insights (`src/lib/insights.ts`) genera observaciones basadas en reglas (no IA) a partir de 12 categorias posibles que cubren emociones, energia, estres, habitos, meditacion, actividad, consistencia, diario, bienestar, nutricion, rachas de imperio y balance financiero. El sistema utiliza `gatherData()` que realiza 15 consultas paralelas a 7 tablas de la base de datos. Los usuarios Free reciben un maximo de 3 insights (vs 5 para Elite), no ven la comparativa semanal (`WeeklyComparison` es null para Free), y las descripciones de insights no incluyen tendencias semanales. En la UI (`insights/page.tsx`), las secciones de comparativa semanal, detalle de bienestar, nutricion, finanzas y rachas de imperio estan bloqueadas con PremiumGate. Las metricas basicas (check-ins, habitos, meditacion, diario, actividad total) son visibles para todos.

3.7 Observaciones

ESTADO: Implementada Completamente

Las observaciones se generan en `src/lib/life-memory/observations.ts` mediante logica pura (sin IA). El sistema construye observaciones de 4 tipos: Etapas (mapeando `LifeStage` objects), Transiciones (mapeando cambios entre etapas), Memorias destacadas (extrayendo contenido real de `FinanceLog.contexto`, `JournalEntry.content` y `DailyCheckin.note`, hasta 10 items), y Patrones (mapeando la salida del detector de patrones cross-imperio). Las memorias extraen texto real del usuario: hasta 5 logs financieros con contexto, 3 entradas de diario (truncadas a 120 caracteres), y 3 notas de check-in (requieren mas de 5 caracteres). Este sistema alimenta tanto la pagina Memoria de Vida como el contexto del Mentor IA.

3.8 Diario

ESTADO: Implementada Completamente

El diario (`src/app/api/journal/route.ts`) implementa CRUD completo con campos: titulo, contenido, mood (1-5 corazones), y gratitud. La validacion requiere que al menos uno de titulo, contenido o gratitud sea no vacio. La creacion otorga +20 XP al imperio Crecimiento con proteccion de carrera via `pg_advisory_xact_lock`. La eliminacion revierte el XP en una transaccion. La pagina del imperio Crecimiento muestra las entradas agrupadas por fecha (Hoy, Ayer, Esta semana, 2 semanas, Mes) con animacion de micro-recompensa. Sin gate de pago: completamente funcional para todos los usuarios.

3.9 Notas

ESTADO: NO IMPLEMENTADA

La funcionalidad "Notas" **no existe como tal** en el codigo. No hay pagina /notas, ni ruta API /api/notes, ni modelo Prisma de notas. La pagina de precios lista "Notas basicas de cada imperio" (Free) y "Notas con mas detalle de cada imperio" (Elite), pero esto se refiere al componente **EmpireTipsSection** que muestra consejos pre-escritos por imperio, no notas creadas por el usuario. Los campos "notes" que existen en wellness, nutrition y timeline son campos de anotacion opcionales dentro de esos formularios, no un sistema de notas independiente. El termino "Notas" en la pagina de precios es engañoso.

3.10 Logros

ESTADO: Implementada Completamente

El sistema de logros (`src/lib/achievements.ts`) contiene **45 logros** (27 visibles + 18 ocultos). Los visibles cubren 6 categorias: Meditacion (5 logros: 1, 10, 30, 100 sesiones), Diario (4: 1, 10, 30, 100), Bienestar (3: 1, 15, 50), Habitos (4: 1, 5, 14 días racha, 30 días racha), Nutricion (3: 1, 15, 50), Finanzas (4: 1, 1 ingreso, 20, 50), Check-in (3: 1, 7, 30), y General (4: 5 imperios activos, 1 cierre, 3 cierres, mas). Los 18 ocultos se revelan al alcanzar 75% de progreso e incluyen hitos como 100 check-ins, 200 diarios, 1 año de uso, equilibrio de imperios, y regresos tras ausencia. El calculo de progreso utiliza 17+ consultas paralelas via `Promise.allSettled`. Sin gate de pago: funcional para todos.

3.11 Imperios (5 imperios)

ESTADO: Implementada Completamente

Los 5 imperios tienen paginas completamente funcionales con CRUD real y datos propios. **Disciplina** (`disciplina/page.tsx`): habitos CRUD con retador diario auto-completable, rachas, y frecuencias. **Mente** (`mente/page.tsx`): meditacion con timer + 4 tecnicas de respiracion guiada (Diafragmatica, Coherencia Cardiaca, Nadi Shodhana, Box Breathing) con animacion visual. **Energía** (`energia/page.tsx`): bienestar (mood, energia, sueno, estres + notas) y nutricion (comidas, agua, calorías, notas). **Riqueza** (`riqueza/page.tsx`): finanzas con mapeo smart keyword-a-categoria, intenciones, contexto, y navegacion por periodos mensuales. **Crecimiento** (`crecimiento/page.tsx`): diario completo con titulo, contenido, mood y gratitud. Cada imperio muestra tips en la parte inferior via `EmpireTipsSection`. Sin gate de pago en ningun CRUD de imperio.

3.12 Recomendaciones

ESTADO: Implementada Completamente (con salvedad)

Las recomendaciones (`src/lib/emotional-state.ts`, funcion `generateRecommendation()`) son un sistema basado en reglas (no IA) que utiliza 4 inputs: energia, foco, estres y consistencia (todos 0-100). La filosofia es "observar, no aconsejar": el sistema nota patrones sin dar instrucciones directas. Existen 8 posibles recomendaciones ("Sin prisa", "Un día a la vez", "Hoy, poco a poco", etc.), pero muchas condiciones retornan cadena vacia (silencio deliberado).

Para Elite, el weekly recap genera recomendaciones mas ricas que incluyen tendencias semanales ("La presion subio esta semana"). La salvedad es que la mayor parte del tiempo el sistema retorna silencio, lo que podria percibirse como una funcionalidad vacia por el usuario.

3.13 Patrones

ESTADO: Implementada Parcialmente (gating visual)

El motor de patrones (**src/lib/patterns/detector.ts**) implementa 5 detectores de correlacion cross-imperio usando logica pura y correlacion de Pearson. Los 5 patrones son: baja energia-gasto impulsivo (finanzas-energia), practica mental-estabilidad financiera (finanzas-mente), estres-cambio financiero (finanzas-estres), sueno-gasto (finanzas-sueno), y crecimiento-estabilidad (finanzas-energia). Requieren minimo 2 semanas de solapamiento, 4 puntos de datos por imperio, y confianza mayor o igual a 0.55. Para usuarios Free, la deteccion se ejecuta en el servidor, pero el UI (**LifePatternsSection.tsx**) muestra solo la primera observacion al 35% de opacidad con el texto "Mas conexiones con el tiempo". Los usuarios Elite ven todas las observaciones con texto completo y etiquetas de imperio.

3.14 Cierre Mensual

ESTADO: Implementada Parcialmente (gating por plan)

El cierre mensual (**src/lib/monthly-closure/digest.ts**) es un sistema sofisticado que agrega datos de 7 tablas en paralelo. Tiene dos fases: Reflexion (texto privado del usuario, "NUNCA enviado a IA") y Resumen. Para usuarios Free, el resumen incluye: reflejo escrito, balance de intenciones (4 tipos de intencion financiera), resumen financiero (ingresos, gastos, top categorias), y ritmo (conteos de actividad de los 5 imperios). Las secciones de Evolucion (comparativa mes-a-mes) y Memorias (extractos de diario, finanzas y check-ins) estan bloqueadas con PremiumGate. En el servidor, **trimDigestForFree()** (lineas 130-136 de la ruta API) elimina evolution (null) y memories (array vacio). La funcionalidad base es completamente funcional.

3.15 Notificaciones

ESTADO: Implementada Parcialmente (3 placebos de 4)

El sistema de notificaciones push tiene una infraestructura de calidad de produccion: integracion FCM real, sistema de gates de 6 pasos (push activado, toggle del tipo, horas silenciosas, cap diario, cooldown por tipo, cap semanal), deduplicacion, limpieza de tokens invalidos, y 21 plantillas de texto. Sin embargo, de los 4 tipos de recordatorios, solo **1 tiene un trigger real que dispare notificaciones**: el tipo "Reflexion" (cron a las 18:00 UTC). Los otros 3 tipos (Check-in, Resumen semanal, Te echamos de menos) tienen toggles funcionales en la UI que persisten estado, pero **ningun codigo llama a sendNotification() para esos tipos**. El propio codigo documenta esto en un comentario de auditoria en NotificationPreferences.tsx. Ademas, los "Recordatorios diarios" por email en Ajustes son explicitamente documentados como PLACEBO en settings/route.ts. El resumen semanal por email (Resend) si funciona correctamente.

3.16 Perfil

ESTADO: Implementada Completamente

El perfil (**src/app/(dashboard)/perfil/page.tsx**) permite editar: avatar (subida con procesamiento cliente de resize/compress jpg/png/webp/gif, validacion servidor), nombre (max 100 chars), pais (max 80), ciudad (max 80), edad (1-150, NumericInput), y bio (max 300 chars). Email, plan y fecha de registro son solo lectura. Muestra badge

de plan (Free o Elite) y fallback con la primera letra del nombre cuando no hay avatar. Sin gate de pago. La validacion se aplica en el servidor (`src/app/api/profile/route.ts`).

3.17 Ajustes

ESTADO: Implementada Parcialmente (1 placebo)

Los ajustes (`src/app/(dashboard)/ajustes/page.tsx`) incluyen: toggle de resumen semanal por email (funcional, consumido por el cron `weekly-recap-sender.ts`), toggle de recordatorios diarios por email (PLACEBO documentado en el codigo), toggle de privacidad de estadisticas (funcional, controla `PrivacyMask` en toda la UI), panel completo de notificaciones push (solo 1 de 4 tipos funcional), enlace a edicion de perfil, gestion de suscripcion via Stripe Portal, cierre de sesion, verificacion de email, y enlaces legales. El componente `SubscriptionManager` muestra el plan actual, la fecha de renovacion para Elite, y el boton de upgrade para Free. El "Recordatorios diarios" esta explicitamente marcado como PLACEBO en `ajustes/page.tsx` (linea 149-150) y `api/settings/route.ts` (linea 54).

4. Auditoria del Plan Elite - Verificacion de Promesas

La pagina de precios ([pricing/page.tsx](#)) lista 8 promesas especificas para Elite, y la pagina elite dedicada ([elite/page.tsx](#)) anade 4 promesas en prosa. Se han verificado todas contra el codigo. A continuacion se presenta el analisis promesa por promesa con las evidencias exactas encontradas.

4.1 Promesas verificadas

Promesa Elite	Estado	Evidencia principal
Conexiones entre tus imperios	IMPLEMENTADA	patterns/detector.ts - 5 detectores cross-imperio con correlacion Pearson
Patrones de vida: lo que se repite	IMPLEMENTADA	Mismo sistema - patron de peso ligera/relevante/profunda
Mentor sin limite diario, con mas contexto	IMPLEMENTADA	limits.ts: Infinity, mentor-context.ts: 5 capas vs basico
Memoria que acumula contexto	IMPLEMENTADA	Emotional State + Life Stages + Silent Memories en contexto Premium
Historial completo de conversaciones	IMPLEMENTADA	threads/route.ts: 10 vs todos, messages: 50 vs sin limite
Notas con mas detalle de cada imperio	PLACEBO	No existe diferenciacion de notas entre planes en ningun archivo
Recomendaciones del mentor completas	IMPLEMENTADA	weekly-recap: tendencias + mentor-context: contexto enriquecido
Observaciones semanales con mas detalle	IMPLEMENTADA	insights.ts: 3 vs 5, comparison null vs real, trend en emotional-state
Cierres mensuales con evolucion	IMPLEMENTADA	trimDigestForFree() elimina evolution y memories para Free

Resultado: 8 de 9 promesas implementadas. 1 placebo completo. La unica promesa no implementada es "Notas con mas detalle de cada imperio", que no tiene equivalente en el codigo. Ningun archivo diferencia notas entre Free y Elite. El componente EmpireTipsSection muestra consejos pre-escritos, no notas del usuario, y la unica diferencia es que Elite ve 1 tip adicional (de tipo PREMIUM) con contenido completo, mientras Free solo ve el titulo. Esto no constituye "mas detalle en las notas" sino "un consejo adicional visible".

4.2 Diferenciacion real del contexto del Mentor Elite

La diferenciacion mas profunda entre Free y Elite reside en el contexto que recibe el Mentor IA. El sistema de contexto ([src/lib/mentor-context.ts](#)) implementa una arquitectura de 5 capas exclusiva para Elite. A continuacion se detalla que datos adicionales recibe Elite respecto a Free.

Fuente de datos	Free	Elite	Lineas mentor-context.ts
Check-ins recientes	2	5	161-166
Rachas de habitos	4	8	168-173
Sesiones de meditacion	1	5	175-180
Entradas de diario	1 (solo titulo)	3 (con contenido)	182-192
Hilos de conversacion	1	3	194-203
Progreso de imperios	No	Todos (5)	205-208
Actividad semanal	No	Meditacion, diario, bienestar, nutricion	210-284
Datos semana anterior	Check-ins	Habitos, meditacion, diario, nutricion	286-304
Datos de onboarding	No	Objetivos, foco, niveles	241-254
Logs de bienestar	No	14 dias: sueno, mood, estres, notas	256-264
Logs financieros	No	30 dias: gasto, mood, contexto	266-274
Estado emocional completo	No	Status, metricas, recomendacion	388-425
Cierres mensuales	No	Todos los registros	427-441
Etapas de vida	No	Etapas 3 meses + transiciones	443-473
Observaciones de patrones	No	Correlaciones cross-imperio	476-565
Silent Memories	No	Hitos rare/very_rare	567-585

5. Conexiones Reales Entre Imperios

Esta seccion analiza todas las conexiones implementadas entre los 5 imperios de VitaZen. Es importante entender que las conexiones no ocurren a nivel de pagina (ninguna pagina de imperio importa componentes de otro imperio), sino a nivel de agregacion de datos en los modulos de `src/lib/`. Los imperios estan silenciados a nivel de recoleccion pero integrados a nivel de analisis. A continuacion se listan todas las conexiones reales con los archivos y funciones exactas.

5.1 Conexiones directas (motor de patrones)

El motor de patrones (`src/lib/patterns/detector.ts`) es el unico modulo que implementa correlaciones estadisticas directas entre pares de imperios. Utiliza el coeficiente de correlacion de Pearson sobre datos agregados semanalmente, con un umbral de confianza de 0.55, minimo 2 semanas de solapamiento y 4 puntos de datos por imperio. Todas las 5 conexiones detectadas involucran al imperio Finanzas.

Conexion	Imperios	Detector	Correlacion	Lineas
Baja energia / Gasto impulsivo	Finanzas - Energia	<code>detectLowEnergyImpulsiveSpending</code>	Negativa	214-250
Practica mental / Estabilidad	Finanzas - Mente	<code>detectMentalPracticeFinancialStability</code>	Positiva	252-288
Estres / Cambio financiero	Finanzas - Energia	<code>detectStressFinancialChange</code>	Bidireccional	290-327
Sueno / Gasto	Finanzas - Energia	<code>detectSleepFinanceConnection</code>	Negativa	329-365
Intencion crecimiento / Calma	Finanzas - Energia	<code>detectGrowthStability</code>	Positiva	367-405

5.2 Conexiones agregadas (modulos analiticos)

Mas alla del motor de patrones, existen multiples conexiones indirectas a traves de modulos que agregan datos de varios imperios simultaneamente.

Modulo	Funcion	Imperios conectados	Archivo
Emotional State Engine	<code>computeEnergy()</code>	General (check-ins) + Energia (wellness sueno)	<code>emotional-state.ts</code>
Emotional State Engine	<code>computeFocus()</code>	General (check-ins) + Mente (meditacion)	<code>emotional-state.ts</code>
Emotional State Engine	<code>computeStress()</code>	General (check-ins) + Energia (wellness estres)	<code>emotional-state.ts</code>
Emotional State Engine	<code>computeConsistency()</code>	General + Disciplina (habitos) + Mente (meditacion)	<code>emotional-state.ts</code>
<code>Insights gatherData()</code>	15 consultas paralelas	Todos los 5 imperios	<code>insights.ts</code> (linea 78)
Monthly Closure <code>computeRhythm()</code>	Conteo de actividad	Todos los 5 imperios (7 tablas)	<code>monthly-closure/digest.ts</code> (linea 184)
Mentor IA	<code>buildMentorContext()</code>	Todos los 5 + cross-imperio patterns	<code>mentor-context.ts</code> (linea 127)
Timeline	Feed cronologico	Mente + Energia + Disciplina + Riqueza	<code>api/timeline/route.ts</code>

Modulo	Funcion	Imperios conectados	Archivo
Momentum	Score consistencia	7 tablas: meditacion, habitos, diario, check-ins, retos, bienestar, nutricion	api/dashboard/momentum/route.ts
Achievements	calculateProgress()	13+ tablas en paralelo, todos los imperios	achievements.ts (linea 150)

5.3 Conexiones NO implementadas

A pesar de la profundidad de las conexiones existentes, hay conexiones logicas que **no estan implementadas** en ningun modulo del sistema. No existen detectores de correlacion para: Disciplina-Energia (habitos vs sueno), Disciplina-Mente (habitos vs meditacion), Mente-Energia (meditacion vs sueno), Energia-Crecimiento (bienestar vs logro de metas), o Disciplina-Riqueza (habitos vs finanzas). Todas las 5 conexiones del motor de patrones involucran a Finanzas. Las correlaciones que no incluyen Finanzas simplemente no existen como modulo de deteccion, aunque la agregacion de datos en el Mentor IA y el Emotional State Engine si cruza estos datos de forma indirecta a traves de las metricas compuestas.

6. Placebos Detectados

Se han identificado 4 placebos en el código: elementos de UI que persisten estado, muestran indicadores visuales de activación, pero cuya funcionalidad de backend nunca se ejecuta. En 3 de los 4 casos, el propio código documenta explícitamente que son placebos.

Placebo	Ubicación	Tipo	Evidencia
Recordatorios diarios (email)	ajustes/page.tsx líneas 184-209	Toggle email	settings/route.ts línea 54: "PLACEBO: stored but NEVER read"
Resumen semanal (push)	NotificationPreferences.tsx líneas 264-273	Toggle push	scheduler.ts línea 113: "toggles exist but no triggers yet"
Te echamos de menos (push)	NotificationPreferences.tsx líneas 276-284	Toggle push	Ningún sendNotification() con type comeback en todo el código
Notas con más detalle (Elite)	pricing/page.tsx línea 166	Promesa de venta	No existe diferenciación de notas entre planes en ningún archivo del repositorio

6.1 Detalle de cada placebo

Recordatorios diarios por email: El toggle en Ajustes persiste el valor `User.dailyReminders` a la base de datos. Muestra un spinner de carga y un checkmark verde al guardar. Sin embargo, ningún cron, job o trigger lee jamás este campo para enviar emails de recordatorio. El propio archivo de la ruta API lo documenta: "PLACEBO: stored but NEVER read by any backend logic". El resumen semanal por email (diferente toggle) sí funciona correctamente vía Resend y el cron `weekly-recap-sender.ts`.

Resumen semanal por push: El toggle de notificaciones push para el resumen semanal guarda la preferencia en `NotificationPreference.weeklyRecap` y pasa los gates del sistema de notificaciones. Pero ningún cron dispara `sendNotification()` con type "weekly_recap". El resumen semanal se envía por email (funcional), no por push. La versión push es fantasma: el usuario cree que la está activando, pero nunca llegará.

Te echamos de menos por push: Similar al anterior. El toggle para recordatorios de regreso (ausencia) persiste estado en `NotificationPreference.comebackReminders`. No existe ningún cron ni trigger que llame a `sendNotification()` con type "comeback" en todo el código. La infraestructura de FCM está lista, pero el disparador no existe.

Notas con más detalle (Elite): A diferencia de los 3 placebos anteriores (que son toggles funcionales sin backend), este es una promesa de venta directamente falsa. La página de precios (línea 166) promete "Notas con más detalle de cada imperio" como beneficio Elite. La auditoría exhaustiva de las 5 páginas de imperio confirma que no existe ninguna diferencia en las notas entre planes Free y Elite. Los campos "notes" en los formularios de bienestar, nutrición y timeline son idénticos para todos los usuarios. El término "notas" en la página de precios se refiere al componente `EmpireTipsSection` (consejos pre-escritos), no a notas creadas por el usuario.

7. Funciones Ocultas (No Anunciadas en Elite)

Se han identificado 11 funcionalidades completamente implementadas y funcionales que **no aparecen mencionadas en ninguna pagina de precios ni en la pagina Elite**. Estas son funciones reales con codigo de produccion que agregan valor significativo a la aplicacion pero que no se utilizan como argumento de venta.

Funcion oculta	Plan	Descripcion	Archivo clave
Weekly Recap (in-app)	Free + Elite	Informe semanal con score bienestar, actividad, top habitos, estado emocional	api/weekly-recap/route.ts
Daily Quotes (300)	Free	300 citas originales en espanol, deterministas, sin repeticion hasta rotacion completa	lib/daily-quotes.ts
Challenges	Free	Retos diarios aleatorios con auto-completado, mapeo a categorias, +25 XP	api/challenges/route.ts
Timeline cross-imperio	Free	Feed cronologico de toda la actividad de 5 imperios, filtrable	api/timeline/route.ts
Respiracion guiada (4 tipos)	Free	Diafragmatica, Coherencia Cardiaca, Nadi Shodhana, Box Breathing con animacion	imperio/mente/page.tsx
Silent Memories	Free	5 tipos de observaciones raras (retorno, temporal, presencia, shift, recurrencia)	api/silent-memory/route.ts
Widget System (5 tipos)	Todos	Widgets iOS/Android: reflection, momentum, checkin, daily_focus, calm_quote	api/widgets/[type]/route.ts
Email Weekly Recap	Free + Elite	Resumen semanal por email con template dark/champagne via Resend	lib/emails/weekly-recap.ts
ReturnTrigger	Free	Mensaje contextual al regresar (1 dia, 2-3 dias, 4-7, 8-14, 15+ dias)	components/dashboard/ReturnTrigger.tsx
Momentum Score	Free	Score 0-100 de consistencia semanal con 7 factores y tendencia	api/dashboard/momentum/route.ts
Privacy Mask	Free	Difuminado de metricas para uso en publico / screen-sharing	hooks/usePrivacy.ts

Es notable que funciones como el sistema de retos, las 4 tecnicas de respiracion guiada, el Momentum Score con 26 consultas paralelas optimizadas, y el sistema de widgets nativos para iOS/Android sean completamente gratuitas y no aparezcan en ningun material de venta. El Daily Quotes con 300 citas originales en espanol con seleccion determinista cross-device es un contenido editorial sustancial que esta completamente oculto. El sistema de Silent Memories con 5 tipos de observaciones raras y control de rareza por servidor es una funcionalidad premium que se ofrece gratis sin mencionar.

8. Vulnerabilidades de Seguridad Identificadas

Durante la auditoria se ha identificado una vulnerabilidad de seguridad relevante que afecta al gating de contenido de pago.

8.1 API de Patrones sin proteccion en servidor

SEVERIDAD: ALTA

El endpoint **GET /api/patterns** (`src/app/api/patterns/route.ts`) **no verifica el plan del usuario**. Cualquier usuario autenticado (incluso Free) que llame directamente a este endpoint recibe **todas las observaciones de patrones** sin truncamiento ni filtrado. La unica proteccion existe en el cliente: el componente `LifePatternsSection.tsx` (linea 291-296) verifica `isPremium` y muestra solo la primera observacion difuminada para Free. Sin embargo, un usuario que inspeccione las llamadas de red o utilice un cliente API podra acceder a todos los patrones cross-imperio sin pagar.

Esto contrasta con otros endpoints como `/api/life-memory` que si aplican `trimDigestForFree()` en el servidor, o `/api/ai/chat` que aplica `checkAILimit()`. La falta de gate en `/api/patterns` es una omision que deberia corregirse antes del lanzamiento agregando una verificacion de plan y un truncamiento similar al de otros endpoints.

9. Componentes con Nomenclatura Enganosa

Se han encontrado 7 componentes con el prefijo "Premium" que **no verifican el plan del usuario ni aplican ningún gate de pago**. Están disponibles para todos los usuarios independientemente de su suscripción. Aunque esto no es un bug funcional, la nomenclatura es engañosa tanto para desarrolladores (que podrían asumir que ya están gated) como para mantenimiento futuro.

Componente	Disponible para	Funcion real
PremiumBlur	Todos	Difuminado generico (sin verificacion de plan, el llamador debe condicionar)
PremiumEmptyState	Todos	Estado vacio generico, no relacionado con suscripcion
PremiumErrorState	Todos	Estado de error generico, no relacionado con suscripcion
PremiumReflection	Todos	Cita diaria de /api/daily-quote, funcional para todos los usuarios
PremiumSkeleton	Todos	Shimmer de carga generico, no relacionado con suscripcion
SilentMemory	Todos	Observacion rara de /api/silent-memory, funcional para todos los usuarios
MicroReward	Todos	Animacion de exito generica, no relacionado con suscripcion

Solo **2** de los 9 componentes con prefijo "Premium" son realmente gates de pago: **PremiumGate** (que reduce opacidad al 40% y muestra enlace a Elite) y **PremiumHistoryGate** (que muestra un whisper "Hay mas aqui" al final de listas). Los demas son componentes genericos con nombres engañosos que deberían renombrarse para evitar confusion.

10. Matriz Final Completa

La siguiente matriz consolida todas las funcionalidades auditadas con su estado de implementacion para cada plan, el archivo principal de evidencia y el estado verificado.

Funcion	Free	Elite	Archivo	Evidencia
Dashboard	Completa	Completa	dashboard/page.tsx	Saludo, EmotionalHero, imperios, check-in status
Check-in	Completa	Completa	checkin/page.tsx + api/checkin/route.ts	6 campos, CRUD, tendencias 14 dias, XP
Habitos	Completa	Completa	api/habits/route.ts	CRUD, frecuencias, rachas, auto-complete
Mentor IA	Completa (15 msg/dia)	Completa (sin limite)	api/ai/chat/route.ts + groq.ts	Diferenciacion de contexto, prompts, params
Memoria de Vida	Parcial (solo etapas)	Completa	api/life-memory/route.ts	Free: etapas. Elite: +transiciones, memorias, patrones
Tu Evolucion	Parcial (3 insights)	Completa (5 + comparativa)	lib/insights.ts	Motor de reglas, 12 categorias
Observaciones	Completa (etapas)	Completa (todas)	lib/life-memory/observations.ts	4 tipos, datos reales de 3 imperios
Diario	Completa	Completa	api/journal/route.ts	CRUD, titulo/contenido/mood/gratitud, +20 XP
Notas	NO EXISTE	NO EXISTE	N/A	El termino refiere a EmpireTips, no a notas de usuario
Logros	Completa (45 logros)	Completa	lib/achievements.ts	27 visibles + 18 ocultos, 13+ consultas paralelas
Imperios (5)	Completa	Completa	imperio/*/page.tsx	CRUD real, retos, respiracion, finanzas, bienestar
Recomendaciones	Completa (basica)	Completa (con tendencias)	lib/emotional-state.ts	8 reglas, frecuentemente silencio
Patrones	Parcial (1 difuminado)	Completa	lib/patterns/detector.ts	5 detectores cross-imperio, Pearson
Cierre Mensual	Parcial	Completa	lib/monthly-closure/digest.ts	Free: reflejo + balance + ritmo. Elite: +evolucion + memorias
Notificaciones	Parcial (1/4 funcional)	Parcial (1/4 funcional)	lib/notifications/service.ts	Solo reflexion tiene trigger real
Perfil	Completa	Completa	api/profile/route.ts	Avatar, nombre, pais, ciudad, edad, bio
Ajustes	Parcial (1 placebo)	Parcial (1 placebo)	ajustes/page.tsx	Recordatorios diarios = PLACEBO

11. Veredicto Final y Recomendaciones

11.1 Veredicto

D) El sistema necesita una revision antes de publicarse.

VitaZen es una aplicacion con una base tecnica solida, una arquitectura de diferenciacion de planes bien estructurada, y un nivel de proteccion contra condiciones de carrera inusual para su etapa. La gran mayoria de las funcionalidades del plan Free estan completamente implementadas (15 de 17), y la diferenciacion Elite esta profundamente integrada en multiples capas de la aplicacion. Sin embargo, existen problemas que requieren atencion antes del lanzamiento a produccion.

11.2 Problemas criticos

- **PLACEBO DE VENTA:** "Notas con mas detalle de cada imperio" esta anunciado como beneficio Elite pero no existe en el codigo. Esto puede constituir publicidad engañosa. Recomendacion: eliminar esta linea de la pagina de precios, o implementar la diferenciacion real.
- **VULNERABILIDAD DE GATING:** El endpoint `/api/patterns` no verifica el plan del usuario. Los datos de patrones cross-imperio (funcionalidad Elite) son accesibles para cualquier usuario Free que llame al endpoint directamente. Recomendacion: agregar verificacion de plan y truncamiento en el servidor, igual que hace `/api/life-memory`.
- **3 PLACEBOS DE UI:** Los toggles de Recordatorios diarios (email), Resumen semanal (push) y Te echamos de menos (push) no disparan ninguna accion. Los usuarios activan estas funciones creyendo que funcionan. Recomendacion: implementar los triggers o eliminar los toggles del UI.

11.3 Problemas menores

- **NOMENCLATURA ENGANOSA:** 7 componentes con prefijo "Premium" no son gates de pago. Recomendacion: renombrar a nombres descriptivos (Skeleton, EmptyState, Reflection, etc.) para evitar confusion en mantenimiento futuro.
- **LIMITES DESCENTRALIZADOS:** Los limites de funcionalidad (15 mensajes, 10 hilos, 3 insights, etc.) estan codificados en sus respectivos archivos, no centralizados. Recomendacion: crear un archivo `constants.ts` unificado.
- **AUDITOR NOTE DESACTUALIZADO:** El comentario en `NotificationPreferences.tsx` (linea 243) indica que el toggle de check-in push no funciona, pero en realidad si existe un cron funcional en `cron/checkin-reminder/route.ts`. Recomendacion: actualizar el comentario de auditoria.
- **FALTA DE CONEXIONES NO-FINANZERAS:** Los 5 detectores de patrones solo conectan con Finanzas. No existen correlaciones para Disciplina-Energia, Disciplina-Mente, Mente-Energia, etc. Recomendacion: evaluar si se desean agregar mas detectores multi-imperio en futuras versiones.

11.4 Fortalezas observadas

- **PROTECCION DE CONCURRENCIA:** Uso extensivo de `pg_advisory_xact_lock` y `SELECT FOR UPDATE` en operaciones criticas (checkout, limites IA, completion de habitos, creacion de diario, eliminacion con reversion de XP).
- **CONTEXT DEL MENTOR IA:** El sistema de 5 capas para Elite (Identidad, Senales, Experiencia, Patrones, Memoria conversacional) con 16 fuentes de datos y deduplicacion de observaciones es una diferenciacion de pago real y profunda.

- **FUNCIONALIDADES OCULTAS DE CALIDAD:** El sistema de Silent Memories, el Momentum Score, los 4 tipos de respiracion guiada, y el sistema de widgets nativos son funcionalidades premium que se ofrecen gratuitamente.
- **SISTEMA DE LOGROS:** 45 logros con 18 ocultos que se revelan progresivamente, con calculo de progreso resiliente via Promise.allSettled, es un sistema de gamificacion maduro y completo.
- **ARQUITECTURA DE PRIVACIDAD:** El sistema PrivacyMask permite difuminar metricas para uso en publico, y el cierre mensual garantiza que las reflexiones del usuario "NUNCA se envian a IA".